

Event Groups

Event Groups

1. Event Groups Overview
2. Core concepts
 - 2.1 How Event Groups Work
 - 2.2 Event Group API
 - 2.3 Wait Mode Comparison
 - 2.4 Bit Operations
3. Project Architecture and Flow
4. Hardware Dependencies
5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 vSetBit0Task() — Set BIT_0
 - 5.3 vSetBit1Task() — Set BIT_1
 - 5.4 vWaitTask() — Wait Synchronization
 - 5.5 app_main() — Main Entry
6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Normal Operation
7. FAQ

1. Event Groups Overview

Event Group is a mechanism in FreeRTOS for **multi-event synchronization**. It uses a 24-bit bitmap to represent multiple events, where tasks can block and wait for **any one** or **all** specified event bits to be set.

Compared to semaphores (which only represent one event), a key advantage of event groups is: **can wait for multiple independent events to all be ready at once**.

Importance and Application Scenarios: Event groups are indispensable in scenarios requiring "waiting for multiple independent conditions to be satisfied simultaneously". Typical applications include:

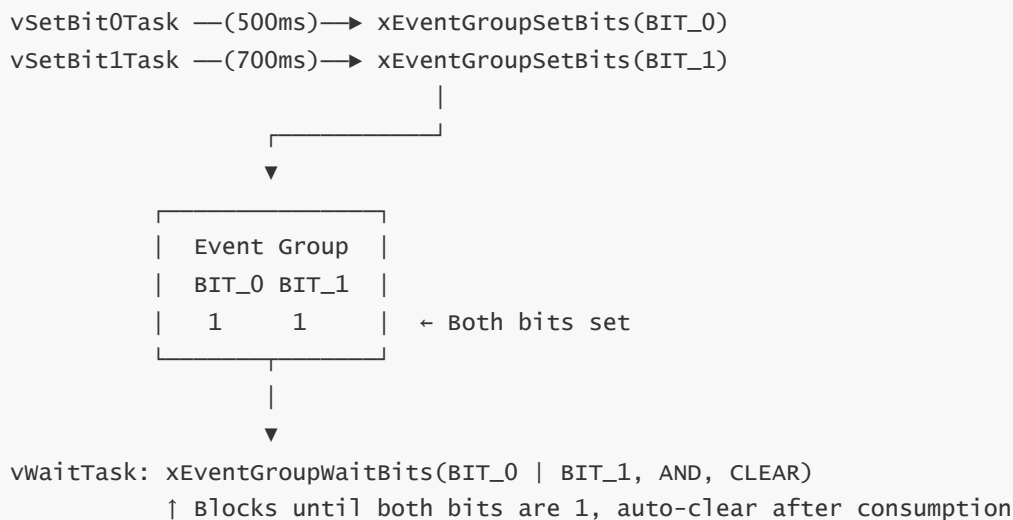
- **Sensor fusion:** Wait for all multiple sensor data to be collected before fusion calculation;
- **Initialization synchronization:** Wait for WiFi, Bluetooth, file system and other modules to all complete initialization before entering main business;
- **Fault detection:** Simultaneously monitor communication timeout, voltage anomaly, watchdog and other fault flags, enter protection state when any triggers;
- **Multi-peripheral coordination:** When multiple peripherals complete tasks at different rates, uniformly trigger next phase operation.

After mastering event groups, you will have the ability to handle complex multi-task synchronization scenarios, which is an important step from single-signal synchronization to system-level task scheduling.

This project demonstrates: two setter tasks run freely at different periods — BIT_0 is set every 500ms, BIT_1 is set every 700ms. Wait task blocks until both bits are ready simultaneously before waking up. After 5 synchronizations, wait task sets BIT_EXIT bit via event group to notify exit, **all inter-task communication completed via event groups** (no global variables involved).

2. Core concepts

2.1 How Event Groups Work



2.2 Event Group API

```
// Create event group
EventGroupHandle_t xEventGroupCreate(void);

// Set event bits
EventBits_t xEventGroupSetBits(
    EventGroupHandle_t xEventGroup,
    // Bits to set (can be multiple, OR together)
    EventBits_t uxBitsToSet
);

// Get current event bits (non-blocking)
EventBits_t xEventGroupGetBits(EventGroupHandle_t xEventGroup);

// wait for event bits
EventBits_t xEventGroupWaitBits(
    EventGroupHandle_t xEventGroup,
    EventBits_t uxBitsToWaitFor, // which bits to wait for
    BaseType_t xClearOnExit,
    BaseType_t xWaitForAllBits, // pdTRUE: AND / pdFALSE: OR
    TickType_t xTicksToWait // Timeout
);
```

2.3 Wait Mode Comparison

<code>xwaitForAllBits</code>	Semantics	Example
<code>pdTRUE</code> (AND)	Returns only when all specified bits are 1	Wait for BIT_0 and BIT_1 simultaneously ready
<code>pdFALSE</code> (OR)	Returns when any specified bit is 1	Wait for BIT_0 or BIT_1 ready

2.4 Bit Operations

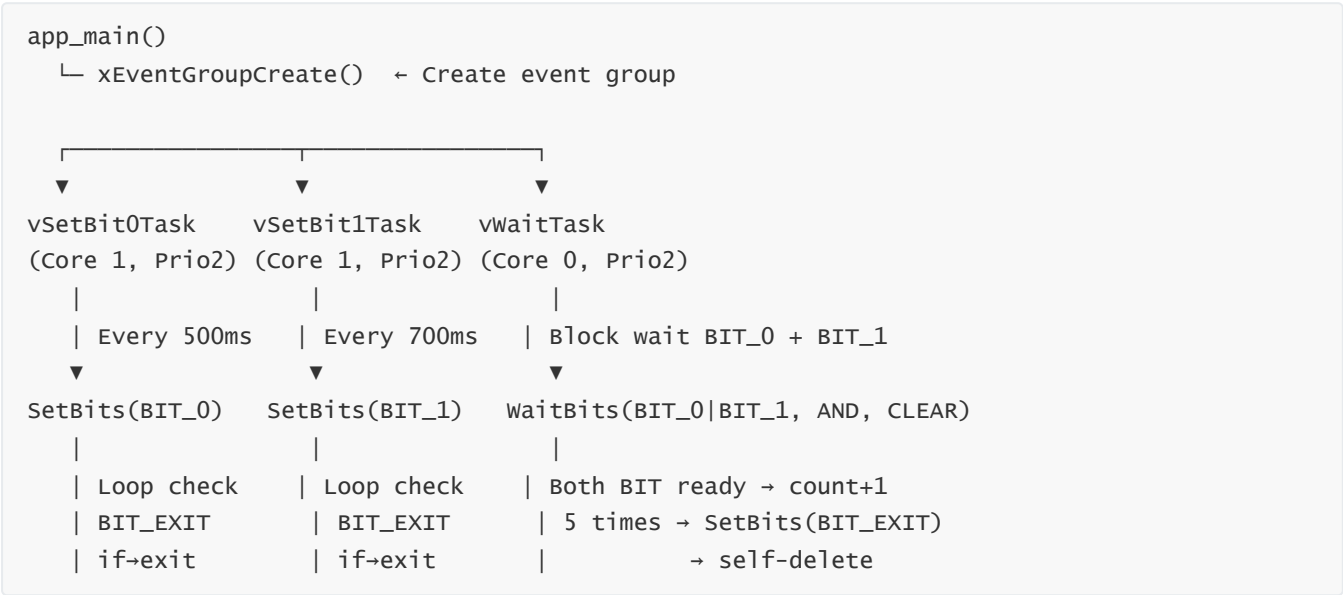
```
#define BIT_0    (1 << 0)    // 0b0001, event bit 0
#define BIT_1    (1 << 1)    // 0b0010, event bit 1
#define BIT_2    (1 << 2)    // 0b0100, event bit 2
#define BIT_3    (1 << 3)    // 0b1000, event bit 3
// ... Up to 24 bits (bit 0 ~ bit 23)

// wait for multiple bits
xEventGroupWaitBits(group, BIT_0 | BIT_1 | BIT_2, ...);
```

This project uses 3 event bits:

Bit	Macro	Setter	Function
BIT_0	<code>(1 << 0)</code>	vSetBit0Task (every 500ms)	Data bit
BIT_1	<code>(1 << 1)</code>	vSetBit1Task (every 700ms)	Data bit
BIT_EXIT	<code>(1 << 2)</code>	vWaitTask (after sync complete)	Exit bit

3. Project Architecture and Flow



Event Bit Parameters:

Bit	Macro	Setter	Description
BIT_0	(1 << 0)	vSetBit0Task (every 500ms)	Data bit
BIT_1	(1 << 1)	vSetBit1Task (every 700ms)	Data bit
BIT_EXIT	(1 << 2)	vWaitTask	Exit signal

Sync Condition: BIT_0 and BIT_1 both ready → waiter count +1 and clear → exit after 5 times.

4. Hardware Dependencies

This project is a **pure software demonstration**, no hardware peripheral dependencies (only serial log output).

Required Components: freertos/FreRTOS, freertos/task, freertos/event_groups, esp_log

5. Source Code Analysis

Full source code: Source Code → ESP-IDF → 4.FreRTOS → 2. RTOS_Inter-task_communication → RTOS_EVENT_GROUP

5.1 Header Files and Macro Configuration

This code section introduces headers required for event groups, defines three event bit macros and global control variables. Exit notification is done via event group bit BIT_EXIT, without using ordinary global variables for flag passing.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_log.h"

static const char *TAG = "EVENT_GROUP_DEMO";

// Event group handle
static EventGroupHandle_t xEventGroup = NULL;

// Define event bits (each occupies 1 bit)
#define BIT_0      (1 << 0)
#define BIT_1      (1 << 1)
#define BIT_EXIT   (1 << 2)

#define MAX_LOOP 5
static int g_loop_count = 0;
```

5.2 vSetBit0Task() — Set BIT_0

This code runs freely at 500ms period, checks exit bit BIT_EXIT in event group via `xEventGroupGetBits()` at the beginning of each loop, self-deletes upon receiving exit signal.

```
static void vSetBit0Task(void *pvParameters)
{
    while (1)
    {
        if (xEventGroupGetBits(xEventGroup) & BIT_EXIT) {
            ESP_LOGI(TAG, "Task1 exit");
            vTaskDelete(NULL);
        }

        xEventGroupSetBits(xEventGroup, BIT_0);
        ESP_LOGI(TAG, "Set BIT0");
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

5.3 vSetBit1Task() — Set BIT_1

This code runs freely at 700ms period, structurally symmetric, checks exit signal via event group.

```
static void vSetBit1Task(void *pvParameters)
{
    while (1)
    {
        if (xEventGroupGetBits(xEventGroup) & BIT_EXIT) {
            ESP_LOGI(TAG, "Task2 exit");
            vTaskDelete(NULL);
        }

        xEventGroupSetBits(xEventGroup, BIT_1);
        ESP_LOGI(TAG, "Set BIT1");
        vTaskDelay(pdMS_TO_TICKS(700));
    }
}
```

5.4 vWaitTask() — Wait Synchronization

This code blocks waiting for BIT_0 and BIT_1 to be ready simultaneously (AND mode, auto-clear), after count reaches MAX_LOOP, sets BIT_EXIT bit via event group to notify all tasks to exit.

```
static void vwaitTask(void *pvParameters)
{
    while (1)
    {
        xEventGroupWaitBits(xEventGroup,
                           BIT_0 | BIT_1,
                           pdTRUE,
```

```

        pdTRUE,
        portMAX_DELAY);

    g_loop_count++;
    ESP_LOGI(TAG, "===== BOTH BITS READY! Count: %d =====", g_loop_count);

    if (g_loop_count >= MAX_LOOP)
    {
        xEventGroupSetBits(xEventGroup, BIT_EXIT);
        ESP_LOGI(TAG, "wait task finish");
        vTaskDelete(NULL);
    }
}
}

```

Why use event group bits instead of global variables for exit notification?

The core concept of event groups is **all inter-task communication is completed via event bits**. If using an ordinary global variable like `bool g_finish` for exit notification, it breaks the synchronization semantics of event groups — tasks both wait for event group bits and poll ordinary variables, mixing two communication mechanisms. After using `BIT_EXIT` event bit, the three tasks coordinate entirely through the event group.

5.5 app_main() — Main Entry

This code creates event group and starts three tasks, then blocks waiting for `BIT_EXIT` exit signal, ensuring `app_main()` only returns after all tasks end, avoiding system log `Returned from app_main()` being inserted into task output stream.

```

void app_main(void)
{
    xEventGroup = xEventGroupCreate();

    if (xEventGroup != NULL)
    {
        ESP_LOGI(TAG, "Event Group created");

        xTaskCreatePinnedToCore(vSetBit0Task, "SetBit0", 2048, NULL, 2, NULL, 1);
        xTaskCreatePinnedToCore(vSetBit1Task, "SetBit1", 2048, NULL, 2, NULL, 1);
        xTaskCreatePinnedToCore(vWaitTask, "Wait", 2048, NULL, 2, NULL, 0);

        xEventGroupWaitBits(xEventGroup, BIT_EXIT, pdFALSE, pdTRUE, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(800));
        vEventGroupDelete(xEventGroup);
    }
}

```

6. Operation Flow

6.1 Compilation and Flashing Flow

For detailed flashing process: 1. Before Development → 3. Build and Flash

6.2 Normal Operation

1. After flashing, power on, observe serial logs
2. BIT_0 is set every 500ms, BIT_1 is set every 700ms (each runs independently)
3. When both BITs are 1 simultaneously, WaitTask wakes up, consumes data bits and count +1

```
I (275) EVENT_GROUP_DEMO: Event Group created
I (275) EVENT_GROUP_DEMO: Set BIT0
I (275) EVENT_GROUP_DEMO: Set BIT1
I (275) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 1 =====
I (775) EVENT_GROUP_DEMO: Set BIT0
I (975) EVENT_GROUP_DEMO: Set BIT1
I (975) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 2 =====
```

4. After 5 synchronizations complete → vWaitTask sets BIT_EXIT → both setters detect exit signal and self-delete

```
I (2375) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 4 =====
I (2775) EVENT_GROUP_DEMO: Set BIT0
I (3075) EVENT_GROUP_DEMO: Set BIT1
I (3075) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 5 =====
I (3075) EVENT_GROUP_DEMO: Wait task finish
I (3275) EVENT_GROUP_DEMO: Task1 exit
I (3775) EVENT_GROUP_DEMO: Task2 exit
```

Log example:

```
I (275) EVENT_GROUP_DEMO: Event Group created
I (275) EVENT_GROUP_DEMO: Set BIT0
I (275) EVENT_GROUP_DEMO: Set BIT1
I (275) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 1 =====
I (775) EVENT_GROUP_DEMO: Set BIT0
I (975) EVENT_GROUP_DEMO: Set BIT1
I (975) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 2 =====
I (1275) EVENT_GROUP_DEMO: Set BIT0
I (1675) EVENT_GROUP_DEMO: Set BIT1
I (1675) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 3 =====
I (1775) EVENT_GROUP_DEMO: Set BIT0
I (2275) EVENT_GROUP_DEMO: Set BIT0
I (2375) EVENT_GROUP_DEMO: Set BIT1
I (2375) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 4 =====
I (2775) EVENT_GROUP_DEMO: Set BIT0
I (3075) EVENT_GROUP_DEMO: Set BIT1
I (3075) EVENT_GROUP_DEMO: ===== BOTH BITS READY! Count: 5 =====
I (3075) EVENT_GROUP_DEMO: wait task finish
I (3275) EVENT_GROUP_DEMO: Task1 exit
I (3775) EVENT_GROUP_DEMO: Task2 exit
I (3875) main_task: Returned from app_main()
```

7. FAQ

Symptom	Cause	Solution
WaitTask never wakes	Event bits never set / bit mask error	Confirm BIT definitions match SetBits calls
Sync count exceeds expectation	Auto-clear not enabled (<code>pdFALSE</code>)	Change to <code>pdTRUE</code> , auto-clear after satisfaction
Only waited for one BIT	<code>xwaitForAllBits</code> set to <code>pdFALSE</code>	Change to <code>pdTRUE</code> (wait for all bits)
Event group creation failed (NULL)	Insufficient memory	Check heap size